

DVL의 $\omega \times r$ 회전 성분 보정

MuJoCo DVL raw 속도에서 **센서 오프셋 때문에 회전만으로 생기는 추가 속도**를 빼서 body 중심 기준 속도에 가깝게 만든다.

1. 핵심 개념

$$\mathbf{v}_{\text{sensor}} = \mathbf{v}_{\text{body}} + \boldsymbol{\omega} \times \mathbf{r}, \quad \mathbf{v}_{\text{body}} \approx \mathbf{v}_{\text{dvl,raw}} - \boldsymbol{\omega} \times \mathbf{r}$$

- $\boldsymbol{\omega}$: 기체 각속도
- $\mathbf{r}$: 기체 중심에서 DVL까지의 위치 벡터
- $\boldsymbol{\omega} \times \mathbf{r}$: 회전만으로 센서 위치에서 생기는 추가 속도

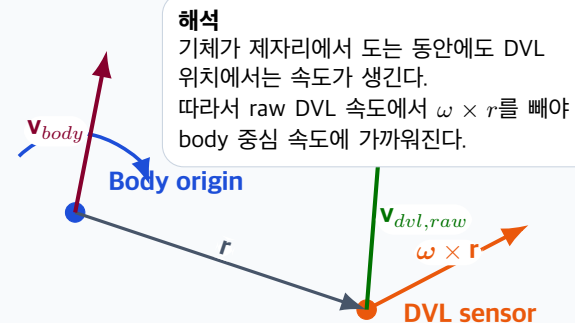
2. 왜 빼야 하나

- DVL은 기체 중심이 아니라 tank_current_scene.xml의 dvl_site처럼 **오프셋된 위치**에 있다.
- 기체가 제자리에서 yaw, pitch, roll만 해도 센서 위치는 원운동한다.
- 이 성분을 그대로 두면 **회전을 병진 속도로 오해**해서 DVL odometry가 틀어진다.

3. 실제 구현

- MuJoCo scene에서 velocimeter dvl_vel_body와 rangefinder dvl_altitude를 읽는다.
- bridge가 DVL 센서측을 body 축으로 돌린다.
- 그 다음 $\text{vel_body} = \text{vel_body} - \text{np.cross}(\text{gyro_bmj}, \text{r_body})$ 로 회전 성분을 뺀다.
- 보정값을 /dvl/velocity, /dvl/odometry, /dvl/data로 publish 한다.

직관 그림



처리 흐름

MuJoCo velocimeter $\rightarrow$ body frame transform $\rightarrow$ subtract $\boldsymbol{\omega} \times \mathbf{r} \rightarrow$ ROS2 / SITL output

코드 포인트

- ros2_bridge.py: `_dvl_velocity_body()`
- ros2_bridge.py: `vel_body = vel_body - np.cross(gyro_bmj, r_body)`
- tank_current_scene.xml: `velocimeter dvl_vel_body`

요약: 회전 때문에 생긴 센서 위치 속도를 빼야 DVL 병진 속도와 odometry drift가 더 정확해진다.